Python Programming — Activity-Based Learning (ABL)

• Ten Selected Case Studies with solutions

- **Student name:** NAKKA HARSHITHA
- **Roll number:** [26000965]
- **Branch:** Computer Science and Engineering (AI & ML)
- **Section:** CSM - 2
- **Course:** Python Programming
- **Institute:** Dr. RVR INSTITUTE OF TECHNOLOGY

• Selected test cases

---

1. Case 13 — ATM Transaction Sequence Simulation
2. Case 18 — Fuel Consumption and Route Comparison
3. Case 28 — Repeated Transaction Detection
4. Case 29 — Password Policy Validation Engine
5. Case 31 — String-Based Examination Answer Evaluation
6. Case 35 — Text Compression Using Character Runs
7. Case 42 — Student Marks Matrix Analysis
8. Case 49 — Tuple-Based Product Catalogue Audit
9. Case 51 — Bus Passenger Occupancy Simulation
10. Case 53 — Food Waste Monitoring System

• Case 13: ATM Transaction Sequence Simulation

1.) Problem statement

Process a sequence of deposits and withdrawals, maintain the running balance, reject invalid transactions, enforce a withdrawal limit, and summarize accepted and rejected transactions.

2.) Objectives

1. Use a list of transaction tuples.
2. Apply validation and conditional branching to deposits and withdrawals.
3. Use a loop to update a running balance and count transaction outcomes.

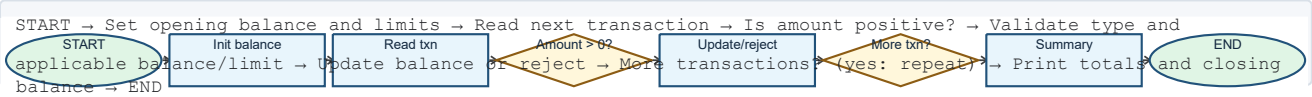
3.) Assumptions

- The opening balance is $5,000 and the maximum single withdrawal is $4,000.
- Amounts must be positive; transaction types are the lowercase strings deposit or withdraw.
- A rejected transaction does not change the balance.

4.) Algorithm

1. Set the current balance to the opening balance and both counters to zero.
2. Visit each transaction in order.
3. Reject a non-positive amount or unknown transaction type.
4. For deposits, add the amount; for withdrawals, check the limit and available balance before subtracting.
5. Update accepted/rejected counts and print the closing summary.

5.) Flowchart



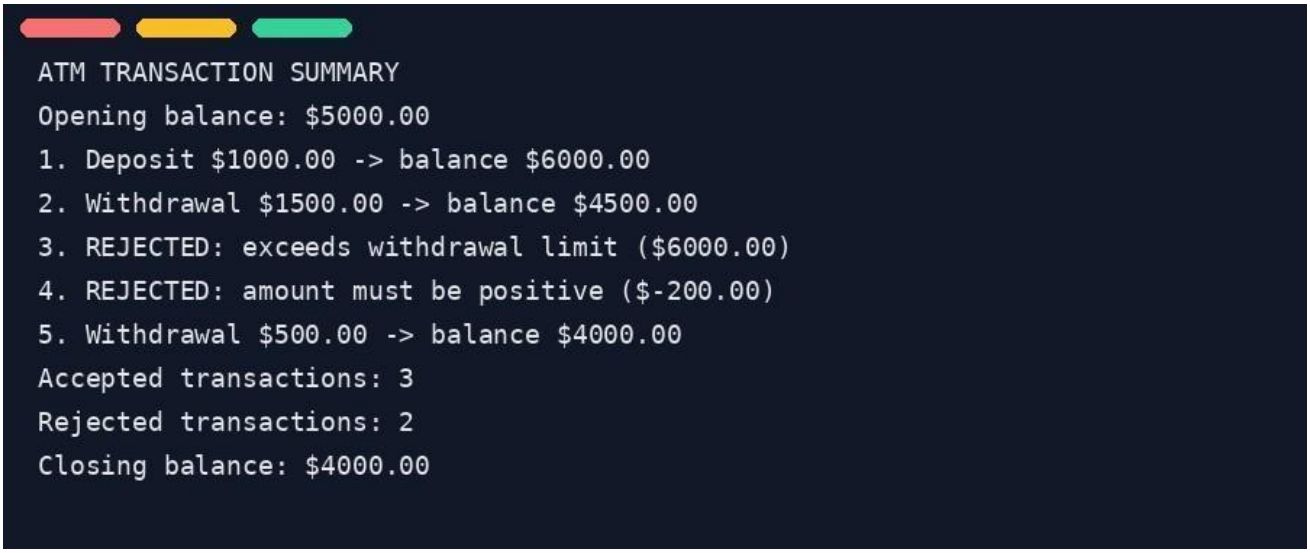
6.) Pseudocode

```
balance ← opening_balance; accepted ← 0; rejected ← 0
FOR each (type, amount) in transactions
  IF amount ≤ 0: reject; rejected ← rejected + 1
  ELSE IF type is deposit: balance ← balance + amount; accepted ← accepted + 1
  ELSE IF type is withdraw AND amount ≤ limit AND amount ≤ balance: subtract; accepted ← accepted + 1
  ELSE: reject; rejected ← rejected + 1
PRINT balance, accepted, rejected
```

7.) Test cases

Test	Input / condition	Expected result
T1	$5,000; deposit $1,000; withdraw $1,500; withdraw $6,000; deposit - $200; withdraw $500	Closing balance $4,000; 3 accepted, 2 rejected
T2	$100; withdraw $150	Insufficient funds; balance remains $100
T3	$100; unknown type, $10	Rejected as unknown type; balance remains $100

8.) Executed output




```
ATM TRANSACTION SUMMARY
Opening balance: $5000.00
1. Deposit $1000.00 -> balance $6000.00
2. Withdrawal $1500.00 -> balance $4500.00
3. REJECTED: exceeds withdrawal limit ($6000.00)
4. REJECTED: amount must be positive ($-200.00)
5. Withdrawal $500.00 -> balance $4000.00
Accepted transactions: 3
Rejected transactions: 2
Closing balance: $4000.00
```

9.) Python source code

```
"""Case 13: ATM transaction sequence simulation."""

INITIAL_BALANCE = 5000.00
MAX_WITHDRAWAL = 4000.00
TRANSACTIONS = [
    ("deposit", 1000.00),
    ("withdraw", 1500.00),
    ("withdraw", 6000.00),
    ("deposit", -200.00),
    ("withdraw", 500.00),
]


def process_transactions(initial_balance, transactions, withdrawal_limit):
    balance = initial_balance
    accepted = 0
    rejected = 0

    print("ATM TRANSACTION SUMMARY")
    print(f"Opening balance: ${balance:.2f}")
    for number, transaction in enumerate(transactions, start=1):
        transaction_type, amount = transaction
        if amount <= 0:
            print(f"{number}. REJECTED: amount must be positive (${amount:.2f})")
            rejected += 1
        elif transaction_type == "deposit":
            balance += amount
            accepted += 1
            print(f"{number}. Deposit ${amount:.2f} -> balance ${balance:.2f}")
        elif transaction_type == "withdraw":
            if amount > withdrawal_limit:
                print(f"{number}. REJECTED: exceeds withdrawal limit (${amount:.2f})")
                rejected += 1
            elif amount > balance:
                print(f"{number}. REJECTED: insufficient funds (${amount:.2f})")
                rejected += 1
            else:
                balance -= amount
                accepted += 1
                print(f"{number}. Withdrawal ${amount:.2f} -> balance ${balance:.2f}")
        else:
            print(f"{number}. REJECTED: unknown transaction type '{transaction_type}'")
            rejected += 1

    print(f"Accepted transactions: {accepted}")
    print(f"Rejected transactions: {rejected}")
    print(f"Closing balance: ${balance:.2f}")
    return balance, accepted, rejected


if __name__ == "__main__":
    process_transactions(INITIAL_BALANCE, TRANSACTIONS, MAX_WITHDRAWAL)
```

10.) Conclusion

The program maintains a correct sequential balance and rejects transactions that fail amount, type, withdrawallimit, or funds checks without altering the account.

• Case 18: Fuel Consumption and Route Comparison

1.) Problem statement

Compare several vehicles for a trip by calculating fuel required with a reserve, checking tank feasibility, estimating cost, and identifying the least-cost feasible vehicle.

2.) Objectives

1. Calculate fuel from distance and mileage.
2. Include a percentage reserve and compare it with tank capacity.
3. Use a list of tuples and select the lowest-cost feasible choice.

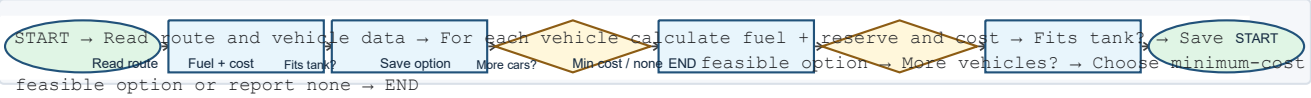
3.) Assumptions

- The trip is 180 km, fuel costs $110 per litre, and reserve is 10% of trip fuel.
- Each vehicle is represented by (name, km per litre, tank capacity in litres).
- The trip is feasible only if its required fuel plus reserve fits in one full tank; refuelling en route is not considered.

4.) Algorithm

1. For each vehicle, validate positive distance and mileage.
2. Compute trip_fuel = distance / mileage.
3. Add the reserve percentage and calculate cost.
4. Compare required fuel with tank capacity; collect feasible vehicles.
5. Print the feasible vehicle with minimum cost, or report none.

5.) Flowchart



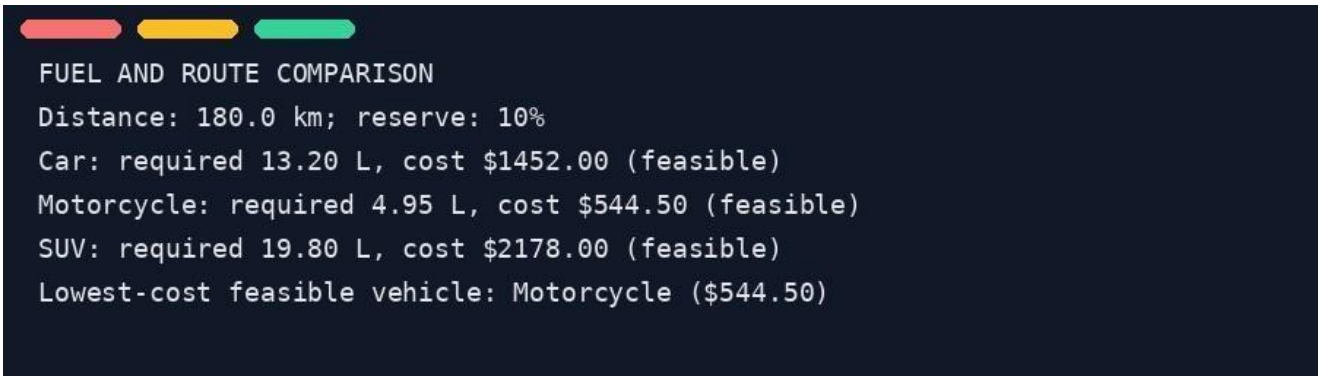
6.) Pseudocode

```
feasible ← empty list
FOR each (name, mileage, tank)
  trip_fuel ← distance / mileage
  required ← trip_fuel × (1 + reserve_percent / 100)
  cost ← required × fuel_price
  IF required ≤ tank: add (name, required, cost) to feasible
IF feasible is not empty: print option with minimum cost
ELSE: print no feasible vehicle
```

7.) Test cases

Test	Input / condition	Expected result
T1	180 km; car 15 km/L and 20 L; bike 40 km/L and 5 L; SUV 10 km/L and 20 L	All fit; motorcycle is least-cost at $544.50
T2	180 km; every tank capacity is 1 L	No vehicle is feasible
T3	Zero mileage for a vehicle	That vehicle is reported as invalid and skipped

8.) Executed output




```
FUEL AND ROUTE COMPARISON
Distance: 180.0 km; reserve: 10%
Car: required 13.20 L, cost $1452.00 (feasible)
Motorcycle: required 4.95 L, cost $544.50 (feasible)
SUV: required 19.80 L, cost $2178.00 (feasible)
Lowest-cost feasible vehicle: Motorcycle ($544.50)
```

9.) Python source code

```
"""Case 18: Fuel consumption and route comparison."""

DISTANCE_KM = 180.0
FUEL_PRICE_PER_LITRE = 110.0
RESERVE_PERCENT = 10.0
VEHICLES = [
    ("Car", 15.0, 20.0),
    ("Motorcycle", 40.0, 5.0),
    ("SUV", 10.0, 20.0),
]


def compare_vehicles(distance, fuel_price, reserve_percent, vehicles):
    feasible = []
    print("FUEL AND ROUTE COMPARISON")
    print(f"Distance: {distance:.1f} km; reserve: {reserve_percent:.0f}%")
    for name, mileage, tank_capacity in vehicles:
        if distance <= 0 or mileage <= 0 or tank_capacity < 0:
            print(f"{name}: invalid vehicle or route data")
            continue
        trip_fuel = distance / mileage
        fuel_with_reserve = trip_fuel * (1 + reserve_percent / 100)
        cost = fuel_with_reserve * fuel_price
        if fuel_with_reserve <= tank_capacity:
            feasible.append((name, fuel_with_reserve, cost))
            print(f"{name}: required {fuel_with_reserve:.2f} L, cost ${cost:.2f} (feasible)")
        else:
            print(f"{name}: required {fuel_with_reserve:.2f} L, tank {tank_capacity:.2f} L (not feasible)")
    if feasible:
        best = min(feasible, key=lambda option: option[2])
        print(f"Lowest-cost feasible vehicle: {best[0]} (${best[2]:.2f})")
        return best
    print("No vehicle is feasible for this trip.")
    return None


if __name__ == "__main__":
    compare_vehicles(DISTANCE_KM, FUEL_PRICE_PER_LITRE, RESERVE_PERCENT, VEHICLES)
```

10.) Conclusion

The comparison accounts for reserve fuel and tank capacity before ranking costs, so the recommended option is both feasible and economical.

• Case 28: Repeated Transaction Detection

1.) Problem statement

Audit a list of transaction identifiers and amounts, report duplicate identifiers, flag amounts at or above a threshold, and identify negative amounts.

2.) Objectives

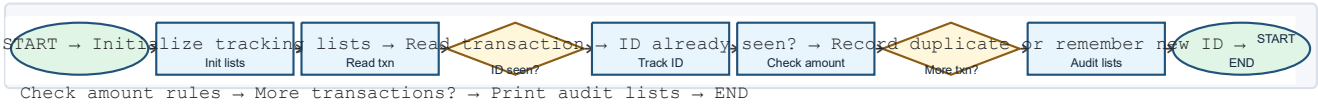
1. Traverse a list of transaction tuples.
2. Use a second list to record identifiers already seen.
3. Apply conditions to identify duplicates, negative amounts, and high-value transactions.

3.) Assumptions

- Transaction identifiers are case-sensitive strings.
- The high-value threshold is $10,000 and an amount equal to the threshold is flagged.
- Every later occurrence of a previously seen identifier is reported as a duplicate. **4.) Algorithm**

1. Create empty lists for seen identifiers, duplicates, and high-value identifiers.
2. For each tuple, check whether its identifier has appeared before.
3. Report and store duplicate identifiers; otherwise add the identifier to seen.
4. Check amount for negativity and against the high-value threshold.
5. Print the duplicate and high-value summaries.

5.) Flowchart



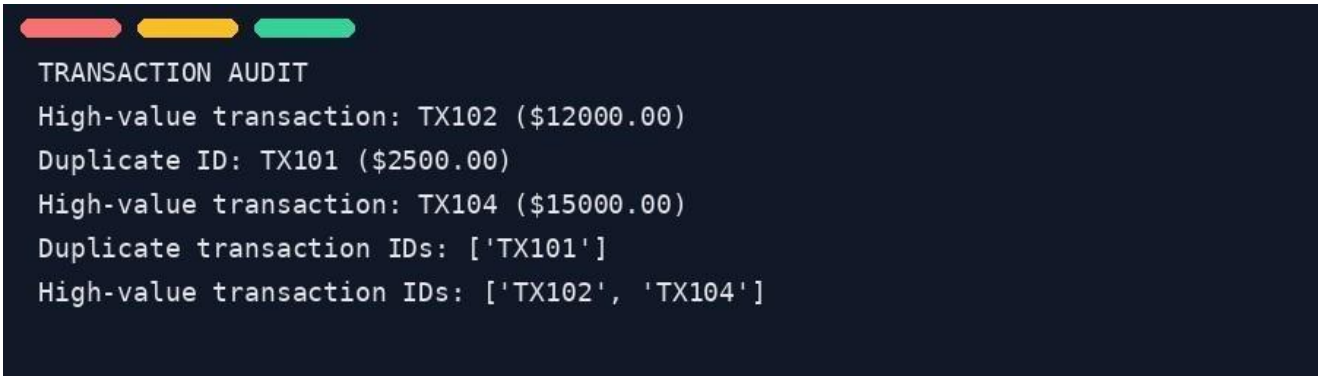
6.) Pseudocode

```
seen ← []; duplicates ← []; high_value ← []
FOR each (id, amount)
  IF id in seen: append id to duplicates
  ELSE: append id to seen
  IF amount < 0: report invalid
  ELSE IF amount ≥ threshold: append id to high_value
PRINT duplicates and high_value
```

7.) Test cases

Test	Input / condition	Expected result
T1	TX101 $2,500; TX102 $12,000; TX101 $2,500; TX104 $15,000	TX101 duplicate; TX102 and TX104 high-value
T2	One transaction of exactly $10,000	Transaction is flagged high-value
T3	Amount -$1	Negative amount is reported invalid

8.) Executed output




```
TRANSACTION AUDIT
High-value transaction: TX102 ($12000.00)
Duplicate ID: TX101 ($2500.00)
High-value transaction: TX104 ($15000.00)
Duplicate transaction IDs: ['TX101']
High-value transaction IDs: ['TX102', 'TX104']
```

9.) Python source code

```
"""Case 28: Repeated transaction detection."""

TRANSACTIONS = [
    ("TX101", 2500.00),
    ("TX102", 12000.00),
    ("TX101", 2500.00),
    ("TX103", 1800.00),
    ("TX104", 15000.00),
]
HIGH_VALUE_LIMIT = 10000.00


def audit_transactions(transactions, high_value_limit):
    seen_ids = []
    duplicates = []
    high_value = []
    print("TRANSACTION AUDIT")
    for transaction_id, amount in transactions:
        if transaction_id in seen_ids:
            duplicates.append(transaction_id)
            print(f"Duplicate ID: {transaction_id} (${amount:.2f})")
        else:
            seen_ids.append(transaction_id)
        if amount < 0:
            print(f"Invalid negative amount: {transaction_id} (${amount:.2f})")
        elif amount >= high_value_limit:
            high_value.append(transaction_id)
            print(f"High-value transaction: {transaction_id} (${amount:.2f})")
    print(f"Duplicate transaction IDs: {duplicates}")
    print(f"High-value transaction IDs: {high_value}")
    return duplicates, high_value


if __name__ == "__main__":
    audit_transactions(TRANSACTIONS, HIGH_VALUE_LIMIT)
```

10.) Conclusion

The audit uses basic list membership and conditional logic to flag repeat IDs and amount anomalies. It is a screening exercise, not a financial fraud determination.

• Case 29: Password Policy Validation Engine

1.) Problem statement

Check example password strings against length, uppercase, lowercase, digit, adjacent-repeat, and prohibitedsubstring rules, and explain each failure. **2.) Objectives**

1. Scan strings one character at a time.
2. Track character-category flags and adjacent repetitions.
3. Return every violated policy rule rather than stopping at the first failure.

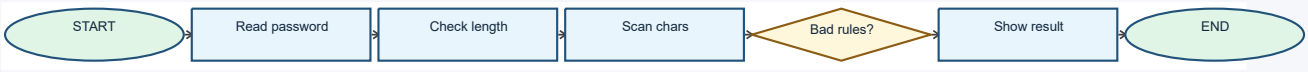
3.) Assumptions

- Minimum length is 8 characters.
- At least one uppercase letter, lowercase letter, and digit are required.
- Identical adjacent characters are disallowed; strings containing password or admin without regard to case are prohibited.

4.) Algorithm

1. Create an empty failure list and set category flags to false.
2. Check string length.
3. Loop through characters to identify uppercase, lowercase, digit, and identical adjacent characters.
4. Check lowercased password for prohibited words.
5. Return VALID when no rules failed; otherwise print each failure.

5.) Flowchart



6.) Pseudocode

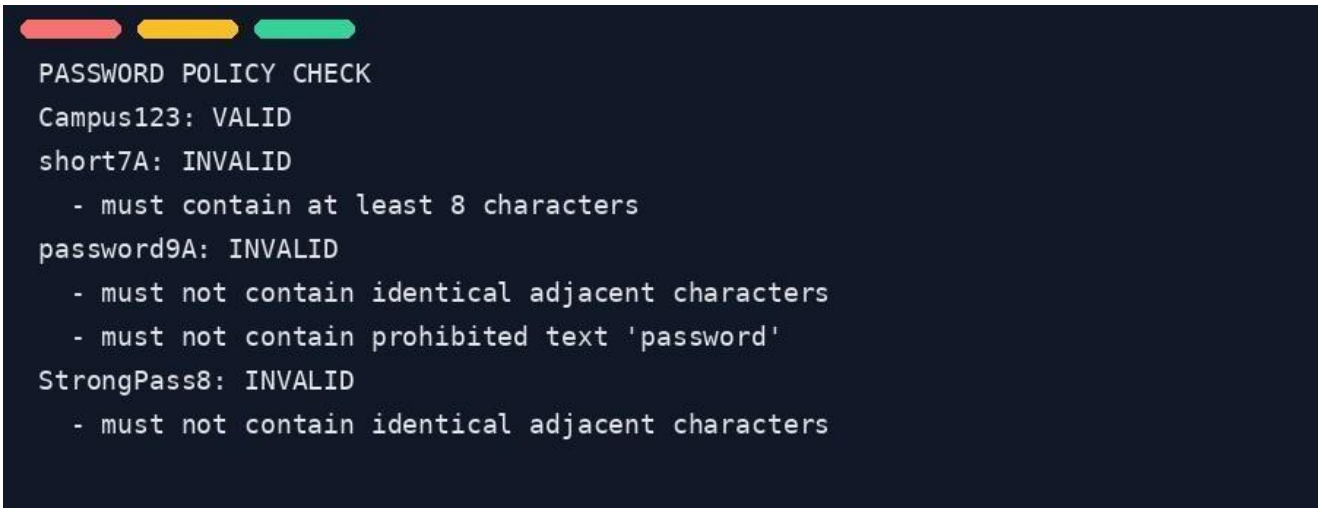
```
failures ← []
IF length(password) < 8: add length failure
FOR each character at index i: update upper/lower/digit flags; compare to previous character
IF a required flag is false: add its failure
IF lower(password) contains prohibited word: add failure
RETURN failures
```

7.) Test cases

Test	Input / condition	Expected result

T1	Campus123	Valid
T2	short7A	Invalid: too short and missing lowercase? (lowercase exists); missing uppercase? has A; missing digit? has 7. Thus only too short
T3	password9A	Invalid: contains prohibited word and repeated adjacent s
T4	StrongPass8	Invalid: repeated adjacent s

8.) Executed output




```
PASSWORD POLICY CHECK
Campus123: VALID
short7A: INVALID
  - must contain at least 8 characters
password9A: INVALID
  - must not contain identical adjacent characters
  - must not contain prohibited text 'password'
StrongPass8: INVALID
  - must not contain identical adjacent characters
```

9.) Python source code

```
"""Case 29: Password policy validation engine."""

PASSWORDS = ["Campus123", "short7A", "password9A", "StrongPass8"]
PROHIBITED_WORDS = ["password", "admin"]


def validate_password(password):
    failures = []
    has_upper = False
    has_lower = False
    has_digit = False
    has_repeated_neighbor = False

    if len(password) < 8:
        failures.append("must contain at least 8 characters")
    for index, character in enumerate(password):
        if character.isupper():
            has_upper = True
        if character.islower():
            has_lower = True
        if character.isdigit():
            has_digit = True
        if index > 0 and character == password[index - 1]:
            has_repeated_neighbor = True
    if not has_upper:
        failures.append("must include an uppercase letter")
    if not has_lower:
        failures.append("must include a lowercase letter")
    if not has_digit:
        failures.append("must include a digit")
    if has_repeated_neighbor:
        failures.append("must not contain identical adjacent characters")
    lower_password = password.lower()
    for word in PROHIBITED_WORDS:
        if word in lower_password:
            failures.append(f"must not contain prohibited text '{word}'")

    return failures


def main():
    print("PASSWORD POLICY CHECK")
    for password in PASSWORDS:
        failures = validate_password(password)
        if failures:
            print(f"{password}: INVALID")
            for reason in failures:
                print(f"  - {reason}")
        else:
            print(f"{password}: VALID")


if __name__ == "__main__":
    main()
```

10.) Conclusion

The checker demonstrates string methods, indexing, loops, and multiple validation flags. It illustrates a classroom policy only and is not a full security-strength assessment.

• Case 31: String-Based Examination Answer Evaluation

1.) Problem statement

Compare each student's answer string with a fixed answer key, count correct, incorrect, and unanswered responses, and calculate marks with negative marking.

2.) Objectives

1. Use string indexing to compare corresponding answers.
2. Distinguish blank responses from wrong responses. 3. Calculate scores using constants and loop counters.

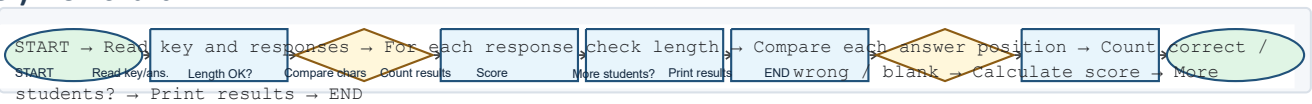
3.) Assumptions

- The key is ABCDC; a hyphen (-) means unanswered.
- A correct response earns 2 marks; a wrong response loses 0.5 marks; blank earns zero.
- Responses must have exactly as many characters as the key.

4.) Algorithm

1. For each response, compare its length with the key length.
2. If lengths differ, report invalid and skip scoring.
3. Otherwise compare characters at corresponding positions and count correct, wrong, and blank answers.
4. Compute score = correct × 2 − wrong × 0.5.
5. Print the counts and score.

5.) Flowchart



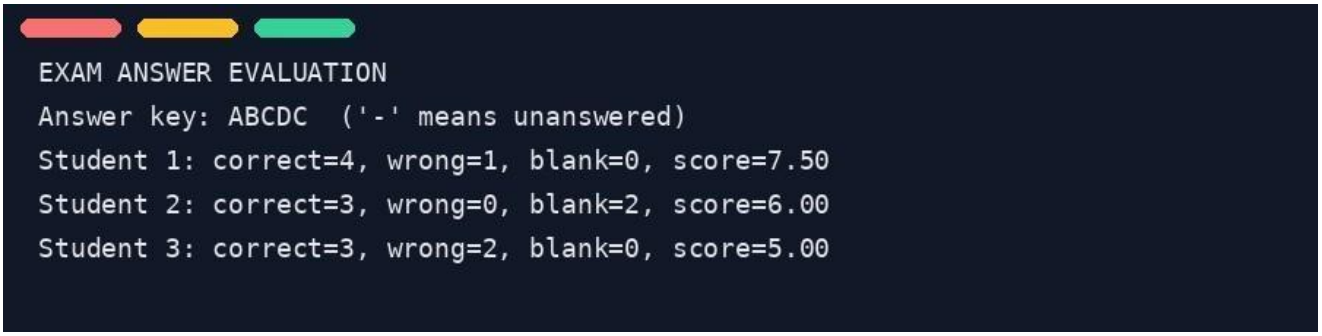
6.) Pseudocode

```
FOR each response
  IF length(response) ≠ length(key): print invalid; continue
  correct ← 0; wrong ← 0; blank ← 0
  FOR each index i: classify response[i] against key[i] or '-'
  score ← correct × marks_correct − wrong × penalty
  PRINT counts and score
```

7.) Test cases

Test	Input / condition	Expected result
T1	ABXDC vs ABCDC	4 correct, 1 wrong, 0 blank; score 7.50
T2	A-CD- vs ABCDC	3 correct, 0 wrong, 2 blank; score 6.00
T3	----- vs ABCDC	0 correct, 0 wrong, 5 blank; score 0
T4	ABCD vs ABCDC	Invalid length; not scored

8.) Executed output




```
EXAM ANSWER EVALUATION
Answer key: ABCDC  ('-' means unanswered)
Student 1: correct=4, wrong=1, blank=0, score=7.50
Student 2: correct=3, wrong=0, blank=2, score=6.00
Student 3: correct=3, wrong=2, blank=0, score=5.00
```

9.) Python source code

```
"""Case 31: String-based examination answer evaluation."""

ANSWER_KEY = "ABCDC"
STUDENT_ANSWERS = ["ABXDC", "A-CD-", "BBCDA"]
MARKS_CORRECT = 2.0
PENALTY_WRONG = 0.5


def evaluate_answers(answer_key, answers, correct_marks, wrong_penalty):
    results = []
    print("EXAM ANSWER EVALUATION")
    print(f"Answer key: {answer_key}  ('-' means unanswered)")
    for student_number, response in enumerate(answers, start=1):
        if len(response) != len(answer_key):
            print(f"Student {student_number}: INVALID (answer count does not match key)")
            continue
        correct = 0
        wrong = 0
        blank = 0
        for index in range(len(answer_key)):
            if response[index] == "-":
                blank += 1
            elif response[index] == answer_key[index]:
                correct += 1
            else:
                wrong += 1
        score = correct * correct_marks - wrong * wrong_penalty
        results.append((student_number, correct, wrong, blank, score))
        print(f"Student {student_number}: correct={correct}, wrong={wrong}, blank={blank}, score={score:.2f}")
    return results



if __name__ == "__main__":
    evaluate_answers(ANSWER_KEY, STUDENT_ANSWERS, MARKS_CORRECT, PENALTY_WRONG)
```

10.) Conclusion

The program gives a transparent score breakdown and treats blank responses separately from incorrect answers.

• Case 35: Text Compression Using Character Runs

1.) Problem statement

Encode consecutive repeated characters as a character followed by its run length, then retain the encoding only if it is shorter than the original string.

2.) Objectives

1. Traverse a string and count consecutive characters.
2. Build an encoded result through concatenation.
3. Compare lengths and fall back to the original if compression is not beneficial.

3.) Assumptions

- Input is an ordinary string; the demonstration uses lowercase letters.
- A run of one character is written once; longer runs are written as character plus decimal count.
- Because the exercise uses alphabetic sample data, encoded digits do not conflict with input digits.

4.) Algorithm

1. Handle empty text as an empty encoding.
2. Set current character to the first character and count to one.
3. Scan remaining characters, increasing count for a match.
4. When the character changes, append the completed run to the encoding and reset the count.
5. Append the final run and choose the shorter of original and encoded strings.

5.) Flowchart



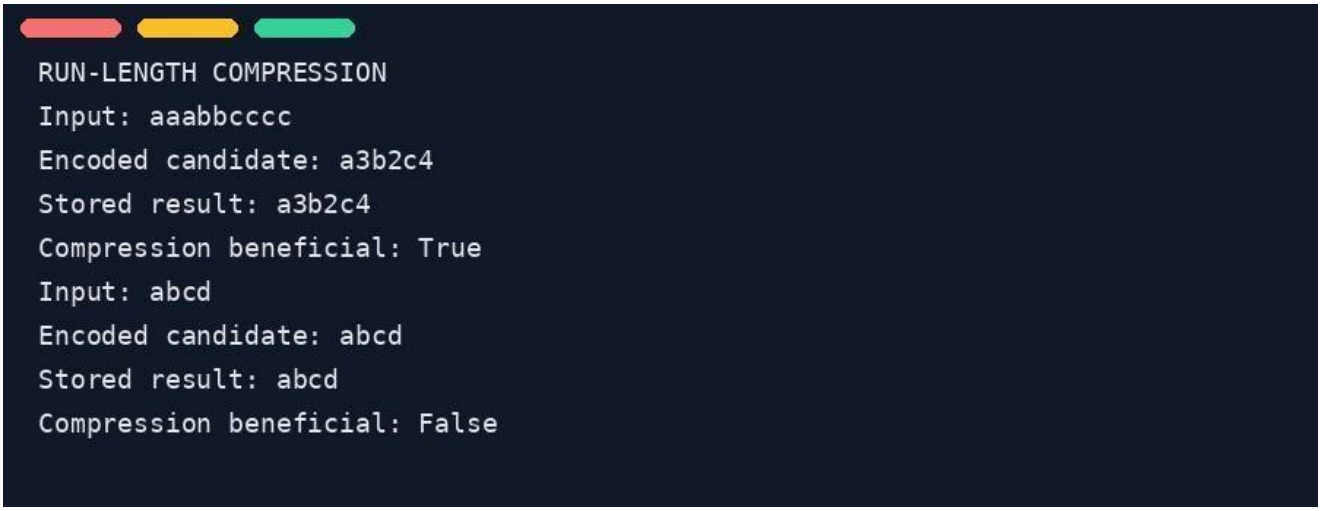
6.) Pseudocode

```
IF text is empty: encoded ← empty
ELSE: current ← first character; count ← 1
FOR each later character
  IF character = current: count ← count + 1
  ELSE: append current and count if count > 1; current ← character; count ← 1
Append final run
IF length(encoded) < length(text): return encoded; ELSE return text
```

7.) Test cases

Test	Input / condition	Expected result
T1	aaabbcccc	Candidate and stored result: a3b2c4; beneficial
T2	abcd	Candidate abcd; original retained because equal length
T3	empty string	Empty result; not beneficial

8.) Executed output




```
RUN-LENGTH COMPRESSION
Input: aaabbcccc
Encoded candidate: a3b2c4
Stored result: a3b2c4
Compression beneficial: True
Input: abcd
Encoded candidate: abcd
Stored result: abcd
Compression beneficial: False
```

9.) Python source code

```
"""Case 35: Text compression using consecutive character runs."""

SAMPLES = ["aaabbcccc", "abcd"]


def encode_runs(text):
    if text == "":
        return ""
    encoded = ""
    current_character = text[0]
    count = 1
    for index in range(1, len(text)):
```

```
        if text[index] == current_character:
            count += 1
        else:
            encoded += current_character
            if count > 1:
                encoded += str(count)
            current_character = text[index]
            count = 1
    encoded += current_character
    if count > 1:
        encoded += str(count)
    return encoded


def compress_if_beneficial(text):
    encoded = encode_runs(text)
    if len(encoded) < len(text):
        return encoded, True
    return text, False


def main():
    print("RUN-LENGTH COMPRESSION")
    for text in SAMPLES:
        result, saved_space = compress_if_beneficial(text)
        print(f"Input: {text}")
        print(f"Encoded candidate: {encode_runs(text)}")
        print(f"Stored result: {result}")
        print(f"Compression beneficial: {saved_space}")


if __name__ == "__main__":
    main()
```

10.) Conclusion

Run counting is implemented with a single pass. Comparing lengths avoids expanding data when run-length encoding would not save space.

• Case 42: Student Marks Matrix Analysis

1.) Problem statement

Analyze a nested list of student marks to calculate each student's total and average, each subject's average, identify lowaverage subjects, and name the top performer.

2.) Objectives

1. Represent marks as a nested list (matrix).
2. Calculate row-wise and column-wise statistics using nested loops.
3. Validate matrix dimensions and mark boundaries.

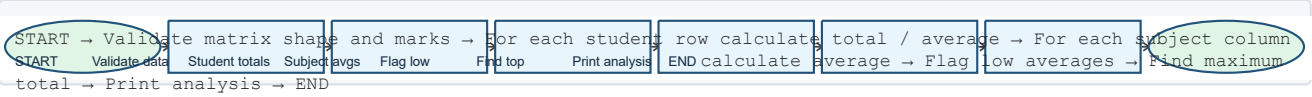
3.) Assumptions

- Each student has one integer mark per listed subject, from 0 to 100 inclusive.
- A subject average below 60 is classified LOW.
- If students tie on total, the first one in the input list is reported as the top performer.

4.) Algorithm

1. Check that the number of student names matches rows, each row matches subjects, and all marks are within range.
2. For every row, calculate total and average.
3. For every subject column, sum marks over rows and calculate average.
4. Compare subject averages with the low threshold.
5. Find the maximum total and print the corresponding student.

5.) Flowchart



6.) Pseudocode

```
VALIDATE row count, column count, and each mark in 0..100
FOR each student row: total ← sum(row); average ← total / subject_count
FOR each subject column: average ← column_sum / student_count
  IF average < threshold: label LOW
Find first student with maximum total
PRINT row results, subject results, and top performer
```

7.) Test cases

Test	Input / condition	Expected result
T1	Provided 3×3 marks matrix	Totals: Asha 247, Bala 235, Charan 277; Charan top
T2	A mark is 101	Raise a validation error
T3	A row has fewer marks than subject count	Raise a matrix-dimension validation error

8.) Executed output

```
STUDENT MARKS MATRIX ANALYSIS
Asha: total=247, average=82.33
Bala: total=235, average=78.33
Charan: total=277, average=92.33
Python average=81.67 (OK)
Maths average=85.00 (OK)
English average=86.33 (OK)
Top performer: Charan (277 marks)
```

```
STUDENT MARKS MATRIX ANALYSIS
Asha: total=247, average=82.33
Bala: total=235, average=78.33
Charan: total=277, average=92.33
Python average=81.67 (OK)
Maths average=85.00 (OK)
English average=86.33 (OK)
Top performer: Charan (277 marks)
```

9.) Python source code

```
"""Case 42: Student marks matrix analysis."""

STUDENTS = ["Asha", "Bala", "Charan"]
SUBJECTS = ["Python", "Maths", "English"]
MARKS = [
    [82, 74, 91],
    [68, 88, 79],
    [95, 93, 89],
]
LOW_AVERAGE_LIMIT = 60.0


def analyze_marks(students, subjects, marks, low_average_limit):
    if len(students) != len(marks) or len(marks) == 0:
        raise ValueError("Each student must have one marks row.")
    for row in marks:
        if len(row) != len(subjects):
            raise ValueError("Each marks row must contain one mark per subject.")
        for mark in row:
            if mark < 0 or mark > 100:
                raise ValueError("Marks must be between 0 and 100.")

    print("STUDENT MARKS MATRIX ANALYSIS")
    totals = []
    for index, student in enumerate(students):
        total = sum(marks[index])
        average = total / len(subjects)
        totals.append(total)
        print(f"{student}: total={total}, average={average:.2f}")

    subject_averages = []
    for column in range(len(subjects)):
        total = 0
        for row in range(len(marks)):
            total += marks[row][column]
        average = total / len(marks)
        subject_averages.append(average)
        status = "LOW" if average < low_average_limit else "OK"
        print(f"{subjects[column]} average={average:.2f} ({status})")

    top_index = totals.index(max(totals))
    print(f"Top performer: {students[top_index]} ({totals[top_index]} marks)")
    return totals, subject_averages, students[top_index]


if __name__ == "__main__":
    analyze_marks(STUDENTS, SUBJECTS, MARKS, LOW_AVERAGE_LIMIT)
```

10.) Conclusion

The matrix is analyzed in both directions and invalid dimensions or marks are rejected before statistics are calculated.

• Case 49: Tuple-Based Product Catalogue Audit

1.) Problem statement

Audit product tuples for invalid fields, calculate valid stock value, and identify items at or below a reorder level.

2.) Objectives

1. Unpack product tuples into named fields.
2. Calculate inventory value as price multiplied by quantity.
3. Use conditions and lists to track invalid and low-stock products.

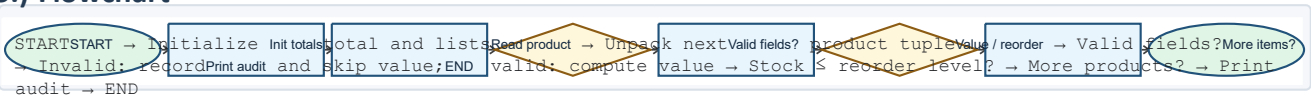
3.) Assumptions

- A product tuple is (identifier, name, unit price, stock quantity).
- Identifiers and names must be non-empty, price must be greater than zero, and stock cannot be negative.
- Reorder is recommended when stock is less than or equal to five units; zero stock is valid and needs reorder.

4.) Algorithm

1. Initialize total value and tracking lists.
2. Unpack each tuple.
3. If fields violate the stated validation rules, report and skip the record.
4. Otherwise compute price × stock and add it to total value.
5. Add products at or below the reorder level to the reorder list.
6. Print totals and lists.

5.) Flowchart



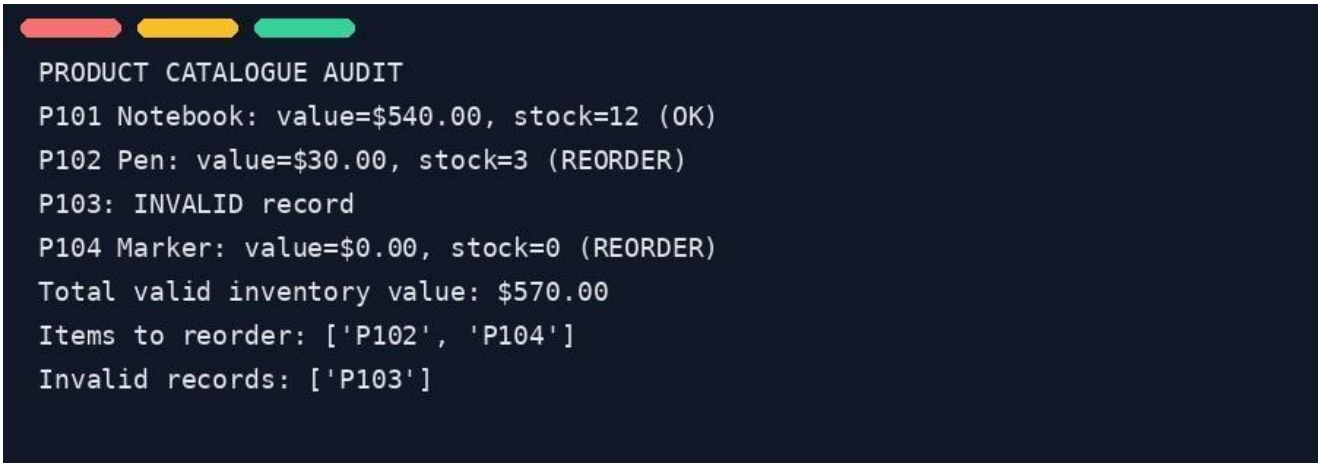
6.) Pseudocode

```
total ← 0; reorder ← []; invalid ← []
FOR each tuple (id, name, price, stock)
  IF id/name empty OR price ≤ 0 OR stock < 0: add id to invalid
  ELSE: total ← total + price × stock; IF stock ≤ reorder_level: add id to reorder
PRINT total, reorder, invalid
```

7.) Test cases

Test	Input / condition	Expected result
T1	Notebook $45×12; Pen $10×3; Folder $0×8; Marker $35×0	Valid total $570; reorder P102 and P104; P103 invalid
T2	Stock 5 at positive price	Valid and marked for reorder
T3	Negative stock	Record invalid and exclude from total

8.) Executed output




```
PRODUCT CATALOGUE AUDIT
P101 Notebook: value=$540.00, stock=12 (OK)
P102 Pen: value=$30.00, stock=3 (REORDER)
P103: INVALID record
P104 Marker: value=$0.00, stock=0 (REORDER)
Total valid inventory value: $570.00
Items to reorder: ['P102', 'P104']
Invalid records: ['P103']
```

9.) Python source code

```
"""Case 49: Tuple-based product catalogue audit."""

# Each immutable tuple is (product_id, name, unit_price, stock_quantity).
PRODUCTS = [
    ("P101", "Notebook", 45.00, 12),
    ("P102", "Pen", 10.00, 3),
    ("P103", "Folder", 0.00, 8),
    ("P104", "Marker", 35.00, 0),
]
REORDER_LEVEL = 5


def audit_catalogue(products, reorder_level):
    total_inventory_value = 0.0
    reorder_items = []
    invalid_records = []
    print("PRODUCT CATALOGUE AUDIT")
    for product in products:
        product_id, name, price, stock = product
        if price <= 0 or stock < 0 or name == "" or product_id == "":
            invalid_records.append(product_id)
            print(f"{product_id}: INVALID record")
            continue
        value = price * stock
        total_inventory_value += value
        status = "REORDER" if stock <= reorder_level else "OK"
        if stock <= reorder_level:
            reorder_items.append(product_id)
        print(f"{product_id} {name}: value=${value:.2f}, stock={stock} ({status})")
    print(f"Total valid inventory value: ${total_inventory_value:.2f}")
    print(f"Items to reorder: {reorder_items}")
    print(f"Invalid records: {invalid_records}")
    return total_inventory_value, reorder_items, invalid_records


if __name__ == "__main__":
    audit_catalogue(PRODUCTS, REORDER_LEVEL)
```

10.) Conclusion

Tuple unpacking keeps product fields readable while validation ensures only valid records contribute to the inventory valuation.

• Case 51: Bus Passenger Occupancy Simulation

1.) Problem statement

Update bus occupancy at successive stops from boarding and alighting counts, enforce capacity, and report passengers who cannot board.

2.) Objectives

1. Represent stops as tuples and process them with a loop.
2. Update occupancy after alighting and before boarding.
3. Limit accepted boarding to available seats and count refused passengers.

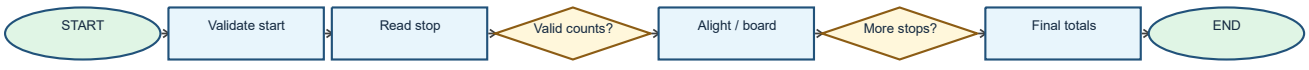
3.) Assumptions

- Bus capacity is 35 and the initial occupancy is 8.
- At a stop, alighting occurs before boarding.
- Negative counts or alighting greater than current occupancy make that stop invalid; excess boarding is refused rather than exceeding capacity.

4.) Algorithm

1. Validate initial occupancy against capacity.
2. For each stop, validate the counts and that alighting does not exceed current passengers.
3. Subtract passengers alighting.
4. Calculate available seats and accept the smaller of requested boarding and available seats.
5. Record refused passengers and print occupancy after each stop.
6. Print final occupancy and total refused.

5.) Flowchart



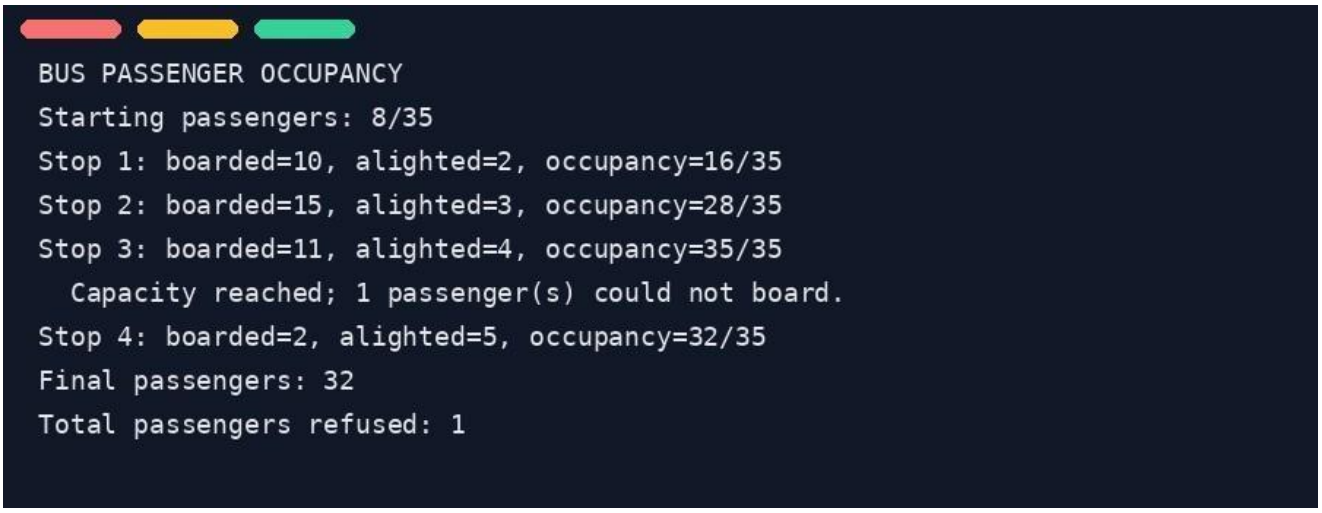

```
passengers ← initial; refused_total ← 0
FOR each (stop, boarding, alighting)
  IF invalid counts or alighting > passengers: report and continue
  passengers ← passengers − alighting
  accepted ← min(boarding, capacity − passengers)
  refused ← boarding − accepted
  passengers ← passengers + accepted
  refused_total ← refused_total + refused
PRINT passengers and refused_total
```

7.) Test cases

Test	Input / condition	Expected result

T1	Initial 8; capacity 35; four provided stops	Final occupancy 32; one passenger refused at Stop 3
T2	Alighting exceeds current occupancy	Stop ignored; occupancy unchanged
T3	Initial occupancy greater than capacity	Raise a validation error

8.) Executed output




```
BUS PASSENGER OCCUPANCY
Starting passengers: 8/35
Stop 1: boarded=10, alighted=2, occupancy=16/35
Stop 2: boarded=15, alighted=3, occupancy=28/35
Stop 3: boarded=11, alighted=4, occupancy=35/35
  Capacity reached; 1 passenger(s) could not board.
Stop 4: boarded=2, alighted=5, occupancy=32/35
Final passengers: 32
Total passengers refused: 1
```

9.) Python source code

```
"""Case 51: Bus passenger occupancy simulation."""

CAPACITY = 35
INITIAL_PASSENGERS = 8
# Tuple for each stop: (stop name, passengers boarding, passengers alighting).
STOPS = [
    ("Stop 1", 10, 2),
    ("Stop 2", 15, 3),
    ("Stop 3", 12, 4),
    ("Stop 4", 2, 5),
]


def simulate_bus(capacity, initial_passengers, stops):
    if initial_passengers < 0 or initial_passengers > capacity:
        raise ValueError("Initial passenger count must be within bus capacity.")
    passengers = initial_passengers
    total_refused = 0
    print("BUS PASSENGER OCCUPANCY")
    print(f"Starting passengers: {passengers}/{capacity}")
    for stop_name, boarding, alighting in stops:
        if boarding < 0 or alighting < 0:
            print(f"{stop_name}: invalid negative count; stop ignored")
            continue
        if alighting > passengers:
            print(f"{stop_name}: invalid alighting count; stop ignored")
            continue
        passengers -= alighting
        available_seats = capacity - passengers
        accepted_boarding = min(boarding, available_seats)
        refused = boarding - accepted_boarding
        passengers += accepted_boarding
        total_refused += refused
        print(f"{stop_name}: boarded={accepted_boarding}, alighted={alighting}, occupancy={passengers}/{capacity}")
        if refused > 0:
            print(f"  Capacity reached; {refused} passenger(s) could not board.")
    print(f"Final passengers: {passengers}")
    print(f"Total passengers refused: {total_refused}")
    return passengers, total_refused


if __name__ == "__main__":
    simulate_bus(CAPACITY, INITIAL_PASSENGERS, STOPS)
```

10.)Conclusion

The simulation preserves the capacity invariant at each stop and reports any unmet boarding demand.

• Case 53: Food Waste Monitoring System

1.) Problem statement

Compare meals prepared and served over multiple days, calculate leftover meals and waste percentages, flag days above a supplied limit, and compute the overall percentage.

2.) Objectives

1. Process daily data tuples in a loop.
2. Calculate leftover quantity and waste percentage.
3. Aggregate counts and classify days against a threshold.

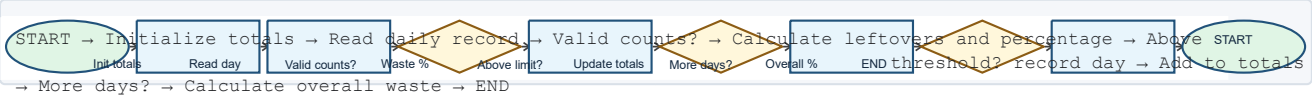
3.) Assumptions

- Each record is (day, meals prepared, meals served).
- Prepared meals must be positive and served meals must be between zero and prepared meals.
- The daily excessive-waste threshold is 15%; only percentages strictly greater than the threshold are flagged.

4.) Algorithm

1. Initialize totals and a list of excessive-waste days.
2. For each day, validate prepared and served values.
3. Compute leftovers = prepared − served and waste percentage = leftovers / prepared × 100.
4. Add valid data to totals and flag days above the limit.
5. Compute and print the overall percentage from totals.

5.) Flowchart



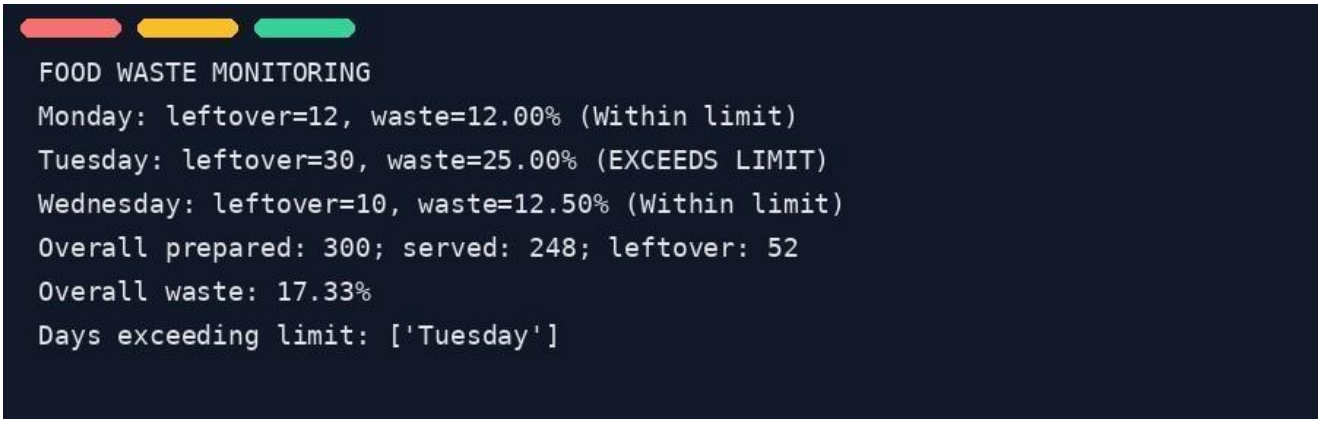
6.) Pseudocode

```
prepared_total ← 0; served_total ← 0; excessive ← []
FOR each (day, prepared, served)
  IF prepared ≤ 0 OR served < 0 OR served > prepared: report invalid; continue
  leftover ← prepared − served
  percentage ← leftover / prepared × 100
  Add prepared and served to totals
  IF percentage > limit: add day to excessive
overall ← (prepared_total − served_total) / prepared_total × 100
PRINT daily and overall results
```

7.) Test cases

Test	Input / condition	Expected result
T1	Mon 100/88; Tue 120/90; Wed 80/70	Tuesday is 25%; overall waste 17.33%
T2	Prepared 100, served 85	Exactly 15%; not flagged because threshold is strict greater-than
T3	Served exceeds prepared	Invalid record; excluded from totals

8.) Executed output




```
FOOD WASTE MONITORING
Monday: leftover=12, waste=12.00% (Within limit)
Tuesday: leftover=30, waste=25.00% (EXCEEDS LIMIT)
Wednesday: leftover=10, waste=12.50% (Within limit)
Overall prepared: 300; served: 248; leftover: 52
Overall waste: 17.33%
Days exceeding limit: ['Tuesday']
```

9.) Python source code

```
"""Case 53: Food waste monitoring system."""

# Each tuple: (day, meals prepared, meals served).
DAILY_MEALS = [
    ("Monday", 100, 88),
    ("Tuesday", 120, 90),
    ("Wednesday", 80, 70),
]
WASTE_LIMIT_PERCENT = 15.0


def analyze_food_waste(daily_meals, waste_limit_percent):
    total_prepared = 0
    total_served = 0
    excessive_days = []
    print("FOOD WASTE MONITORING")
    for day, prepared, served in daily_meals:
        if prepared <= 0 or served < 0 or served > prepared:
            print(f"{day}: invalid meal counts")
            continue
        leftover = prepared - served
        waste_percent = leftover / prepared * 100
        total_prepared += prepared
        total_served += served
        status = "EXCEEDS LIMIT" if waste_percent > waste_limit_percent else "Within limit"
        if waste_percent > waste_limit_percent:
            excessive_days.append(day)
        print(f"{day}: leftover={leftover}, waste={waste_percent:.2f}% ({status})")
    overall_leftover = total_prepared - total_served
    if total_prepared > 0:
        overall_percent = overall_leftover / total_prepared * 100
    else:
        overall_percent = 0.0
    print(f"Overall prepared: {total_prepared}; served: {total_served}; leftover: {overall_leftover}")
    print(f"Overall waste: {overall_percent:.2f}%")
    print(f"Days exceeding limit: {excessive_days}")
    return overall_percent, excessive_days
```

```
if __name__ == "__main__":
    analyze_food_waste(DAILY_MEALS, WASTE_LIMIT_PERCENT)
```

10.) Conclusion

The program measures both daily and aggregate waste, while validating records to avoid impossible meal counts.